Custom Models and Training

CSCI-P 556

ZORAN TIGANJ

Today

- Custom Models and Training with TensorFlow (Chapter 12)
  - Quick tour of TensorFlow
  - Autodiff
  - Custom loss function, layers, activation functions, initializers, regularizers, training loops
- Loading and Preprocessing Data with TensorFlow (Chapter 13)

A Quick Tour of TensorFlow

- TensorFlow is a powerful library for numerical computation, particularly well suited and fine-tuned for large-scale Machine Learning.
  - Its core is very similar to NumPy, but with GPU support.
  - It supports distributed computing (across multiple devices and servers).
  - It includes a kind of just-in-time (JIT) compiler that allows it to optimize computations for speed and memory usage. It works by extracting the computation graph from a Python function, then optimizing it.
  - Computation graphs can be exported to a portable format, so you can train a TensorFlow model in one environment (e.g., using Python on Linux) and run it in another (e.g., using Java on an Android device).
  - It implements autodiff and provides some excellent optimizers, such as RMSProp and Nadam, so you can easily minimize all sorts of loss functions.

TensorFlow's Python API

`tf.keras` `tf.estimator`	High-level Deep Learning APIs

`tf.nn`
`tf.losses`
`tf.metrics`
`tf.optimizers`
`tf.train`
`tf.initializers`
	Low-level Deep Learning APIs

`tf.GradientTape` `tf.gradients()`	Autodiff

`tf.data`
`tf.feature_column`
`tf.audio`
`tf.image`
`tf.io`
`tf.queue`
	I/O and preprocessing

`tf.summary`	Visualization with TensorBoard

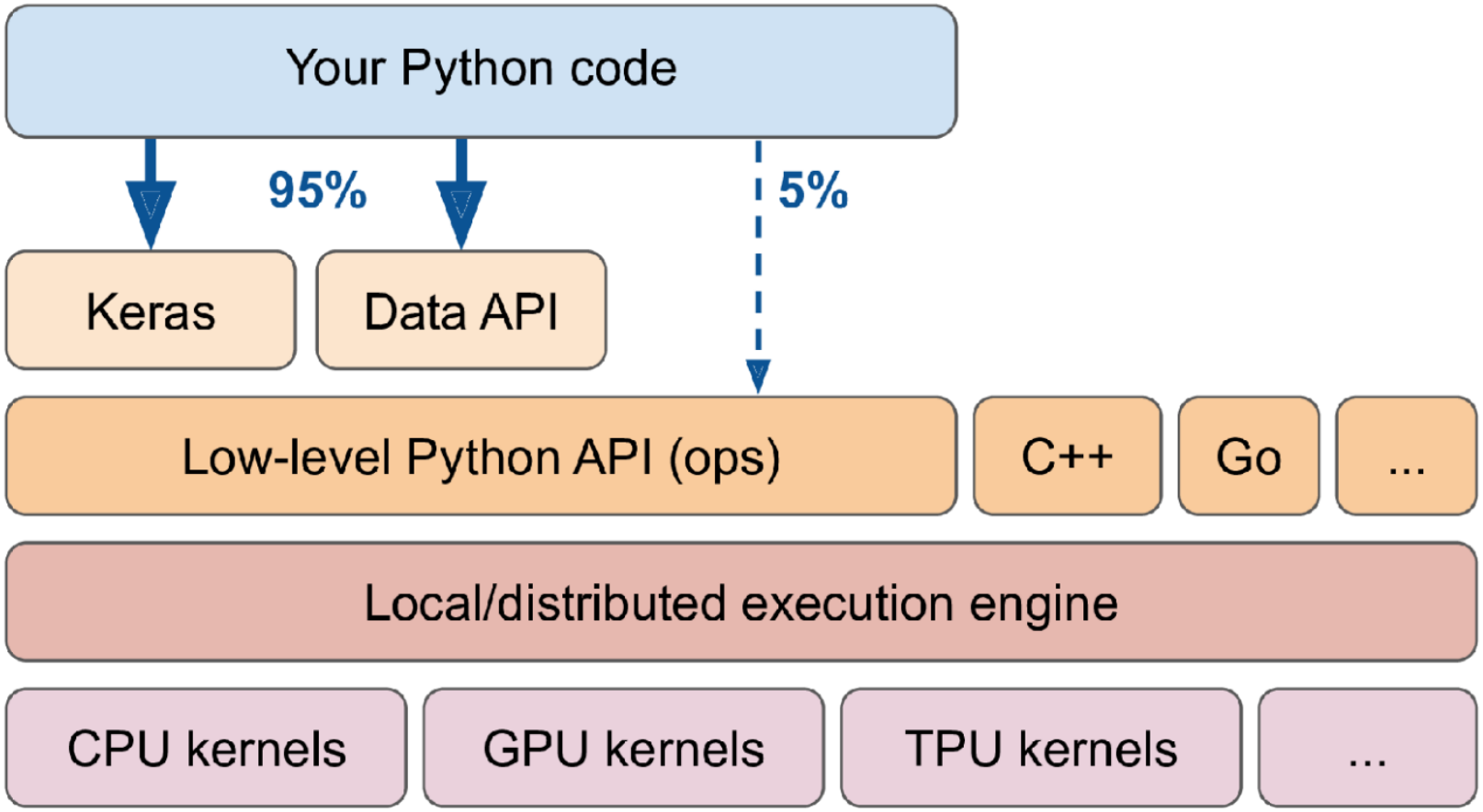
`tf.distribute`
`tf.saved_model`
`tf.autograph`
`tf.graph_util`
`tf.lite`
`tf.quantization`
`tf.tpu`
`tf.xla`
	Deployment and optimization

`tf.lookup`
`tf.nest`
`tf.ragged`
`tf.sets`
`tf.sparse`
`tf.strings`
	Special data structures

`tf.math`
`tf.linalg`
`tf.signal`
`tf.random`
`tf.bitwise`
	Mathematics, including linear algebra and signal processing

`tf.compat` `tf.config` *& more*	Miscellaneous

TensorFlow's architecture



TensorFlow: simple example

```
>>> tf.constant([[1., 2., 3.], [4., 5., 6.]]) # matrix
<tf.Tensor: id=0, shape=(2, 3), dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
>>> tf.constant(42) # scalar
<tf.Tensor: id=1, shape=(), dtype=int32, numpy=42>
```

```
>>> t = tf.constant([[1., 2., 3.], [4., 5., 6.]])
>>> t.shape
TensorShape([2, 3])
>>> t.dtype
tf.float32
```

```
>>> t[:, 1:]
<tf.Tensor: id=5, shape=(2, 2), dtype=float32, numpy=
array([[2., 3.],
       [5., 6.]], dtype=float32)>
```

```
>>> t + 10
<tf.Tensor: id=18, shape=(2, 3), dtype=float32, numpy=
array([[11., 12., 13.],
       [14., 15., 16.]], dtype=float32)>
>>> tf.square(t)
<tf.Tensor: id=20, shape=(2, 3), dtype=float32, numpy=
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)>
>>> t @ tf.transpose(t)
```

Custom Loss Functions

```
def huber_fn(y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < 1
    squared_loss = tf.square(error) / 2
    linear_loss  = tf.abs(error) - 0.5
    return tf.where(is_small_error, squared_loss, linear_loss)
```

```
model.compile(loss=huber_fn, optimizer="nadam")
model.fit(X_train, y_train, [...])
```

For better performance, you should use a vectorized implementation, as in this example. Moreover, if you want to benefit from TensorFlow's graph features, you should use only TensorFlow operations.

Custom Activation Functions, Initializers, Regularizers, and Constraints

```
def my_softplus(z): # return value is just tf.nn.softplus(z)
    return tf.math.log(tf.exp(z) + 1.0)

def my_glorot_initializer(shape, dtype=tf.float32):
    stddev = tf.sqrt(2. / (shape[0] + shape[1]))
    return tf.random.normal(shape, stddev=stddev, dtype=dtype)

def my_l1_regularizer(weights):
    return tf.reduce_sum(tf.abs(0.01 * weights))

def my_positive_weights(weights): # return value is just
tf.nn.relu(weights)
    return tf.where(weights < 0., tf.zeros_like(weights), weights)
```

```
layer = keras.layers.Dense(30, activation=my_softplus,
                           kernel_initializer=my_glorot_initializer,
                           kernel_regularizer=my_l1_regularizer,
                           kernel_constraint=my_positive_weights)
```

Custom Layers

- If you want to create a custom layer without any weights, the simplest option is to write a function and wrap it in a keras.layers.Lambda layer.
- For example, the following layer will apply the exponential function to its inputs

```
exponential_layer = keras.layers.Lambda(lambda x: tf.exp(x))
```

Computing Gradients Using Autodiff

- To understand how to use autodiff to compute gradients automatically, let's consider a simple toy function

```
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2
```

- At point (5, 3), what is the partial derivative wrt w1? 36
- At point (5, 3), what is the partial derivative wrt w1? 10
- In a neural network finding partial derivatives analytically would be almost impossible task.

Computing Gradients Using Autodiff

- One solution could be to compute an approximation of each partial derivative by measuring how much the function's output changes when you tweak the corresponding parameter:

```
>>> w1, w2 = 5, 3
>>> eps = 1e-6
>>> (f(w1 + eps, w2) - f(w1, w2)) / eps
36.000003007075065
>>> (f(w1, w2 + eps) - f(w1, w2)) / eps
10.000000003174137
```

Computing Gradients Using Autodiff

- Let's compute gradients using TensorFlow:

```
w1, w2 = tf.Variable(5.), tf.Variable(3.)
with tf.GradientTape() as tape:
    z = f(w1, w2)

gradients = tape.gradient(z, [w1, w2])
```

- We first define two variables w1 and w2, then we create a tf.GradientTape context that will automatically record every operation that involves a variable, and finally we ask this tape to compute the gradients of the result z with regard to both variables [w1, w2].

```
>>> gradients
[<tf.Tensor: id=828234, shape=(), dtype=float32, numpy=36.0>,
 <tf.Tensor: id=828229, shape=(), dtype=float32, numpy=10.0>]
```

Computing Gradients Using Autodiff

- The tape is automatically erased immediately after you call its gradient() method, so you will get an exception if you try to call gradient() twice:

```
with tf.GradientTape() as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1) # => tensor 36.0
dz_dw2 = tape.gradient(z, w2) # RuntimeError!
```

- If you need to call gradient() more than once, you must make the tape persistent and delete it each time you are done with it to free resources:

```
with tf.GradientTape(persistent=True) as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1) # => tensor 36.0
dz_dw2 = tape.gradient(z, w2) # => tensor 10.0, works fine now!
del tape
```

Computing Gradients Using Autodiff

- In some cases you may want to stop gradients from backpropagating through some part of your neural network.
- To do this, you must use the tf.stop_gradient() function.
- The function returns its inputs during the forward pass (like tf.identity()), but it does not let gradients through during backpropagation (it acts like a constant):

```
def f(w1, w2):
    return 3 * w1 ** 2 + tf.stop_gradient(2 * w1 * w2)

with tf.GradientTape() as tape:
    z = f(w1, w2) # same result as without stop_gradient()

gradients = tape.gradient(z, [w1, w2])
```

Computing Gradients Using Autodiff

- In some cases you may want to stop gradients from backpropagating through some part of your neural network.
- To do this, you must use the tf.stop_gradient() function.
- The function returns its inputs during the forward pass (like tf.identity()), but it does not let gradients through during backpropagation (it acts like a constant):

```
def f(w1, w2):
    return 3 * w1 ** 2 + tf.stop_gradient(2 * w1 * w2)

with tf.GradientTape() as tape:
    z = f(w1, w2) # same result as without stop_gradient()

gradients = tape.gradient(z, [w1, w2]) # => returns [tensor 30., None]
```

Custom Training Loops

- First, let's build a simple model.

```
l2_reg = keras.regularizers.l2(0.05)
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="elu",
kernel_initializer="he_normal",
                       kernel_regularizer=l2_reg),
    keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

Custom Training Loops

- Next, let's create a tiny function that will randomly sample a batch of instances from the training set (see Chapter 13 in the book to learn about the Data API, which offers a much better alternative):

```
def random_batch(X, y, batch_size=32):
    idx = np.random.randint(len(X), size=batch_size)
    return X[idx], y[idx]
```

Custom Training Loops

- Let's also define a function that will display the training status, including the number of steps, the total number of steps, the mean loss since the start of the epoch:

```
def print_status_bar(iteration, total, loss, metrics=None):
    metrics = " - ".join(["{}: {:.4f}".format(m.name, m.result())
                          for m in [loss] + (metrics or [])])
    end = "" if iteration < total else "\n"
    print("\r{}/{} - ".format(iteration, total) + metrics,
          end=end)
```

Custom Training Loops

- Next, we need to define some hyperparameters and choose the optimizer, the loss function, and the metrics

```
n_epochs = 5
batch_size = 32
n_steps = len(X_train) // batch_size
optimizer = keras.optimizers.Nadam(lr=0.01)
loss_fn = keras.losses.mean_squared_error
mean_loss = keras.metrics.Mean()
metrics = [keras.metrics.MeanAbsoluteError()]
```

Custom Training Loops

- And now we are ready to build the custom loop!

```
for epoch in range(1, n_epochs + 1):
    print("Epoch {}/{}".format(epoch, n_epochs))
    for step in range(1, n_steps + 1):
        X_batch, y_batch = random_batch(X_train_scaled, y_train)
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients,
model.trainable_variables))
        mean_loss(loss)
        for metric in metrics:
            metric(y_batch, y_pred)
        print_status_bar(step * batch_size, len(y_train), mean_loss,
metrics)
    print_status_bar(len(y_train), len(y_train), mean_loss, metrics)
    for metric in [mean_loss] + metrics:
        metric.reset_states()
```

Loading and Preprocessing Data with TensorFlow (chapter 13)

- So far we have used only datasets that fit in memory, but Deep Learning systems are often trained on very large datasets that will not fit in RAM.
- Ingesting a large dataset and preprocessing it efficiently can be tricky to implement with other Deep Learning libraries, but TensorFlow makes it easy thanks to the Data API: you just create a dataset object, and tell it where to get the data and how to transform it.
- TensorFlow takes care of all the implementation details, such as multithreading, queuing, batching, and prefetching.

The Data API

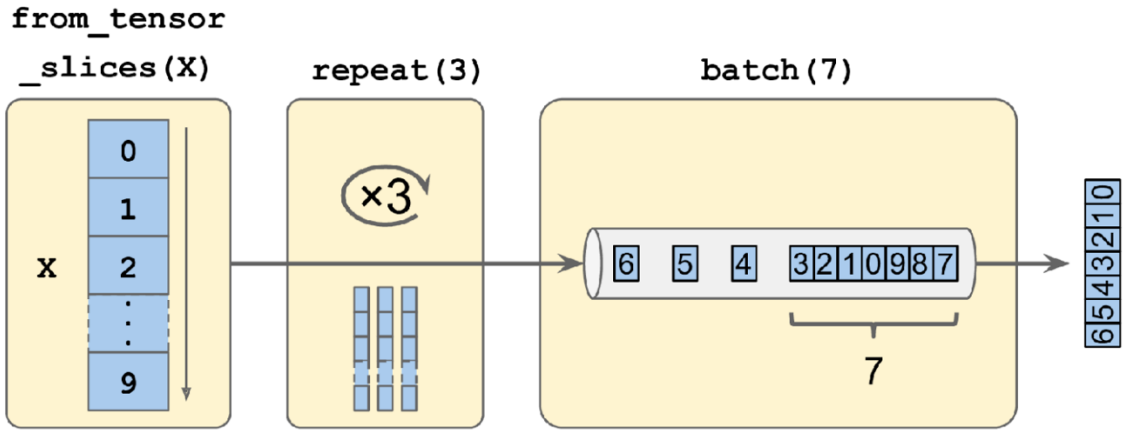
- The whole Data API revolves around the concept of a dataset: sequence of data items.
- Let's create a simple dataset:

```
>>> X = tf.range(10)  # any data tensor
>>> dataset = tf.data.Dataset.from_tensor_slices(X)
>>> dataset
<TensorSliceDataset shapes: (), types: tf.int32>
>>> for item in dataset:
...     print(item)
...
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
tf.Tensor(2, shape=(), dtype=int32)
[...]
tf.Tensor(9, shape=(), dtype=int32)
```

The Data API

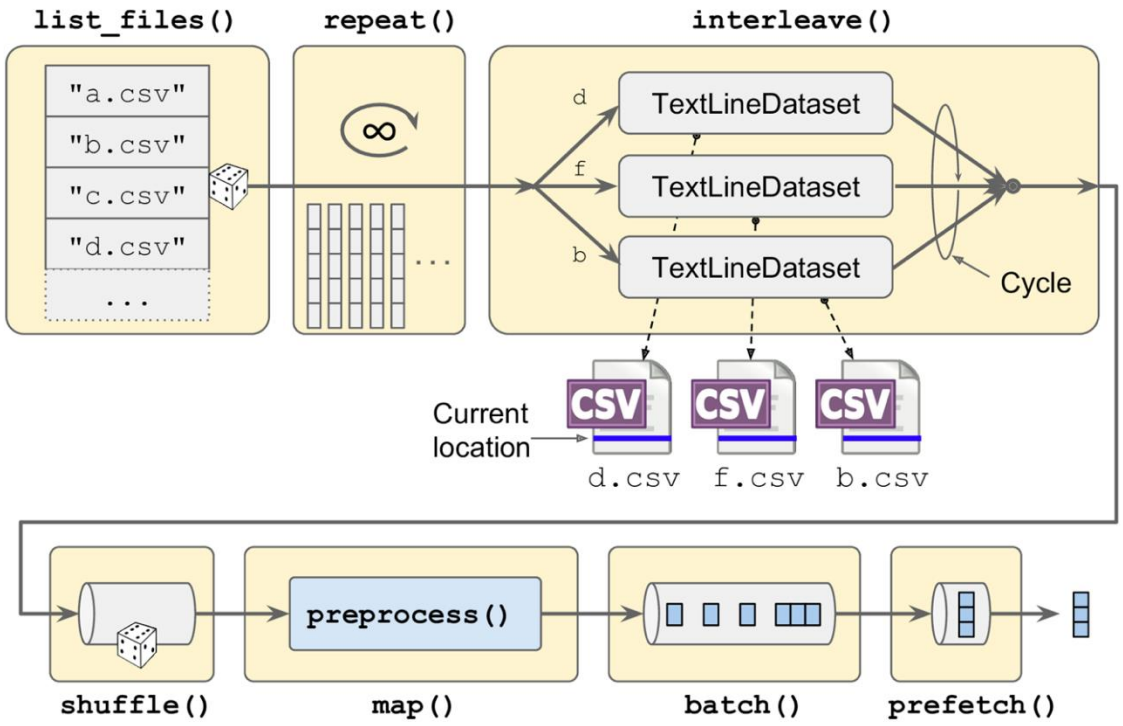
- Once you have a dataset, you can apply all sorts of transformations to it by calling its transformation methods.

```
>>> dataset = dataset.repeat(3).batch(7)
>>> for item in dataset:
...     print(item)
...
tf.Tensor([0 1 2 3 4 5 6], shape=(7,), dtype=int32)
tf.Tensor([7 8 9 0 1 2 3], shape=(7,), dtype=int32)
tf.Tensor([4 5 6 7 8 9 0], shape=(7,), dtype=int32)
tf.Tensor([1 2 3 4 5 6 7], shape=(7,), dtype=int32)
tf.Tensor([8 9], shape=(2,), dtype=int32)
```

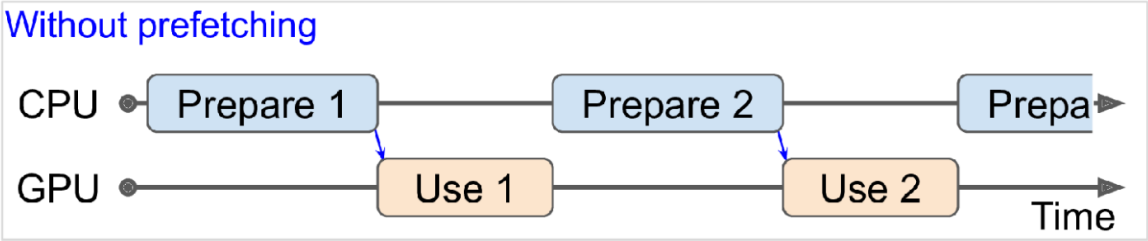


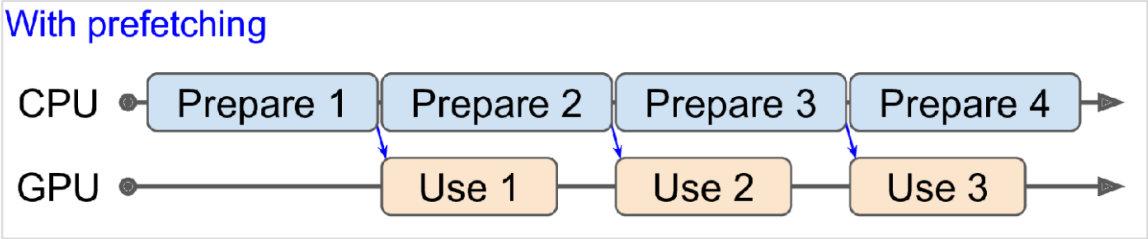
The Data API

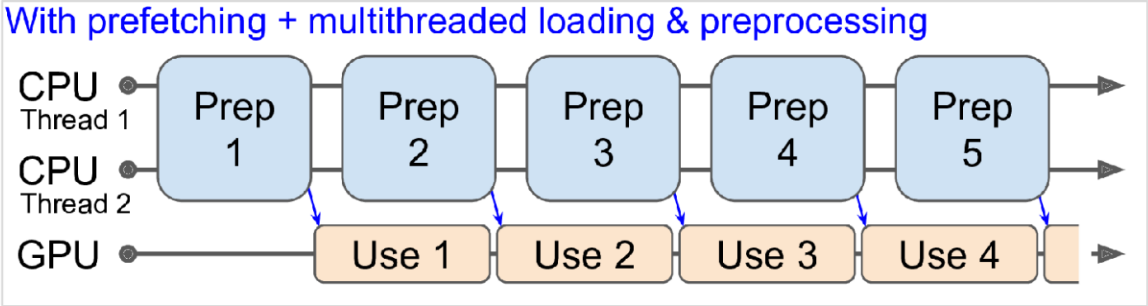
```
def csv_reader_dataset(filepaths, repeat=1, n_readers=5,
                       n_read_threads=None, shuffle_buffer_size=10000,
                       n_parse_threads=5, batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths)
    dataset = dataset.interleave(
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers, num_parallel_calls=n_read_threads)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
    dataset = dataset.shuffle(shuffle_buffer_size).repeat(repeat)
    return dataset.batch(batch_size).prefetch(1)
```



Prefetching







Preprocessing the Input Features

- Here is how you can implement a standardization layer using a Lambda layer.

```
means = np.mean(X_train, axis=0, keepdims=True)
stds = np.std(X_train, axis=0, keepdims=True)
eps = keras.backend.epsilon()
model = keras.models.Sequential([
    keras.layers.Lambda(lambda inputs: (inputs - means) / (stds + eps)),
    [...] # other layers
])
```

Preprocessing the Input Features

- However, you may prefer to use a nice self-contained custom layer (much like Scikit-Learn's StandardScaler), rather than having global variables like means and stds dangling around:

```
class Standardization(keras.layers.Layer):
    def adapt(self, data_sample):
        self.means_ = np.mean(data_sample, axis=0, keepdims=True)
        self.stds_ = np.std(data_sample, axis=0, keepdims=True)
    def call(self, inputs):
        return (inputs - self.means_) / (self.stds_ +
keras.backend.epsilon())
```

Preprocessing the Input Features

- Before you can use this standardization layer, you will need to adapt it to your dataset by calling the adapt() method and passing it a data sample.

```
std_layer = Standardization()
std_layer.adapt(data_sample)
```

Preprocessing the Input Features

- Next, you can use this preprocessing layer like a normal layer:

```
model = keras.Sequential()
model.add(std_layer)
[...] # create the rest of the model
model.compile([...])
model.fit([...])
```

Keras Preprocessing Layers: The Normalization Layer

```
norm_layer = tf.keras.layers.Normalization()
model = tf.keras.models.Sequential([
    norm_layer,
    tf.keras.layers.Dense(1)
])
model.compile(loss="mse", optimizer=tf.keras.optimizers.SGD(learning_rate=2e-3))
norm_layer.adapt(X_train)  # computes the mean and variance of every feature
model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=5)
```

Keras Preprocessing Layers: The Discretization Layer

- The Discretization layer's goal is to transform a numerical feature into a categorical feature by mapping value ranges (called bins) to categories.
- Fixed bin boundaries

```
>>> age = tf.constant([[10.], [93.], [57.], [18.], [37.], [5.]])
>>> discretize_layer = tf.keras.layers.Discretization(bin_boundaries=[18., 50.])
>>> age_categories = discretize_layer(age)
>>> age_categories
<tf.Tensor: shape=(6, 1), dtype=int64, numpy=array([[0],[2],[2],[1],[1],[0]])>
```

Keras Preprocessing Layers: The Discretization Layer

- The Discretization layer's goal is to transform a numerical feature into a categorical feature by mapping value ranges (called bins) to categories.
- Fixes bin number:

```
>>> discretize_layer = tf.keras.layers.Discretization(num_bins=3)
>>> discretize_layer.adapt(age)
>>> age_categories = discretize_layer(age)
>>> age_categories
<tf.Tensor: shape=(6, 1), dtype=int64, numpy=array([[1],[2],[2],[1],[2],[0]])>
```

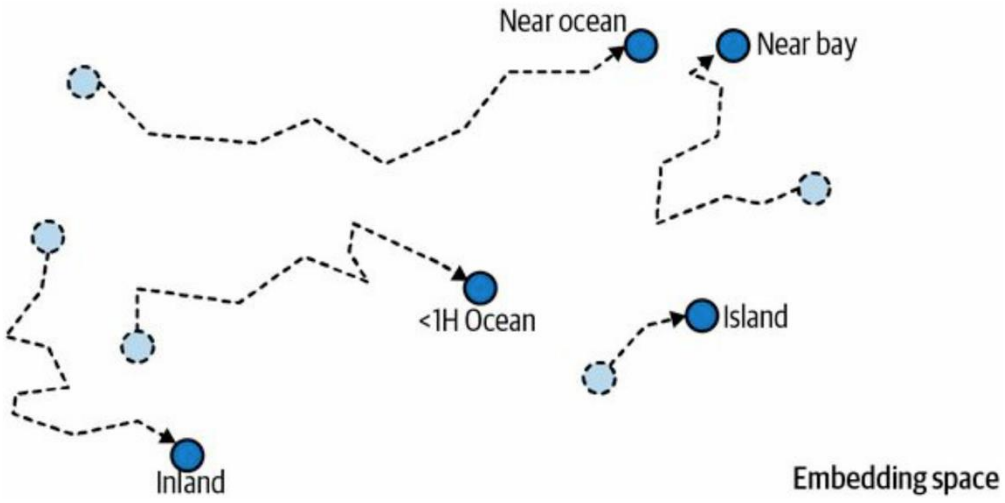
Keras Preprocessing Layers: The CategoryEncoding Layer

- When there are only a few categories (e.g., less than a dozen or two), then one-hot encoding is often a good option.

```
>>> onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3)
>>> onehot_layer(age_categories)
<tf.Tensor: shape=(6, 3), dtype=float32, numpy=
array([[0., 1., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.]], dtype=float32)>
```

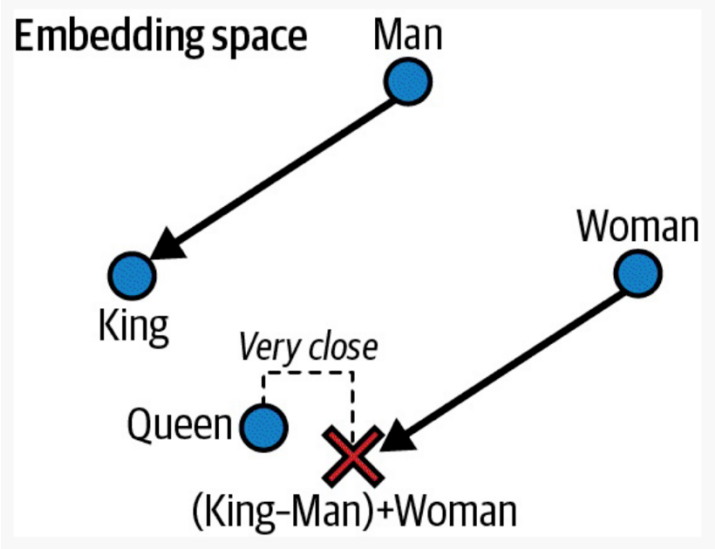
Encoding Categorical Features Using Embeddings

- If there are 50,000 possible categories, then one-hot encoding would produce a 50,000-dimensional sparse vector (i.e., containing mostly zeros).
- In contrast, an embedding would be a comparatively small dense vector; for example, with just 100 dimensions.
- In deep learning, embeddings are usually initialized randomly, and they are then trained by gradient descent, along with the other model parameters.



Word embeddings

- Not only will embeddings generally be useful representations for the task at hand, but quite often these same embeddings can be reused successfully for other tasks.
- The most common example of this is word embeddings



King – Man + Woman = Queen

Keras embedding layer

```
>>> tf.random.set_seed(42)
>>> embedding_layer = tf.keras.layers.Embedding(input_dim=5, output_dim=2)
>>> embedding_layer(np.array([2, 4, 2]))
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[-0.04663396,  0.01846724],
       [-0.02736737, -0.02768031],
       [-0.04663396,  0.01846724]], dtype=float32)>
```

Keras embedding layer

```
>>> tf.random.set_seed(42)
>>> ocean_prox = ["<1H OCEAN", "INLAND", "NEAR OCEAN", "NEAR BAY", "ISLAND"]
>>> str_lookup_layer = tf.keras.layers.StringLookup()
>>> str_lookup_layer.adapt(ocean_prox)
>>> lookup_and_embed = tf.keras.Sequential([
...     str_lookup_layer,
...     tf.keras.layers.Embedding(input_dim=str_lookup_layer.vocabulary_size(),
...                               output_dim=2)
... ])
...
>>> lookup_and_embed(np.array([["<1H OCEAN"], ["ISLAND"], ["<1H OCEAN"]]))
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[-0.01896119,  0.02223358],
       [ 0.02401174,  0.03724445],
       [-0.01896119,  0.02223358]], dtype=float32)>
```

Using Pretrained Language Model Components

- The TensorFlow Hub library makes it easy to reuse pretrained model components in your own models, for text, image, audio, and more (see also Hugging Face for more trained models).
- These model components are called modules.
- For example, some powerful pretrained language models are available.
- The most powerful are quite large (several gigabytes), so for a quick example let's use the nnlm-en-dim50 module, version 2, which is a fairly basic module that takes raw text as input and outputs 50-dimensional sentence embeddings.

```
>>> import tensorflow_hub as hub
>>> hub_layer = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim50/2")
>>> sentence_embeddings = hub_layer(tf.constant(["To be", "Not to be"]))
>>> sentence_embeddings.numpy().round(2)
array([[-0.25,  0.28,  0.01,  0.1 ,  [...] ,  0.05,  0.31],
       [-0.2 ,  0.2 , -0.08,  0.02,  [...] , -0.04,  0.15]], dtype=float32)
```

Next time

- Convolutional Neural Networks (Chapter 14)